



Module 02 — เทคโนโลยีและเครื่องมือสำหรับ Big Data

ผู้สอน: **พศ.ดร. เอกรัฐ รัชกาญจน์**

คณะสถิติประยุกต์ · สถาบันบัณฑิตพัฒนบริหารศาสตร์ (นิด้า)

หัวข้อบรรยาย

- **ส่วนที่ 1:** ภาพรวม Distributed Computing และ Hadoop Ecosystem
- **ส่วนที่ 2:** Apache Spark — RDD, DataFrame, Performance
- **ส่วนที่ 3:** แพลตฟอร์มยุคใหม่ — Databricks, BigQuery

01

ภาพรวม Distributed Computing และ Hadoop Ecosystem

Distributed Computing คืออะไร?

- ระบบที่ประกอบด้วย **หลายเครื่อง** เชื่อมต่อกันผ่าน network
- ทำงานร่วมกัน เสมือนเป็นเครื่องเดียว
- ใช้ทรัพยากรร่วมกัน (CPU, Memory, Storage)

ลักษณะสำคัญ

คุณสมบัติ	คำอธิบาย
Concurrency	หลายโปรเซสทำงานพร้อมกัน
No shared clock	แต่ละ node มี clock ของตัวเอง
Fault tolerance	รองรับความล้มเหลวของบาง node
Scalability	เพิ่ม/ลด node ได้ตามปริมาณข้อมูล
Transparency	ผู้ใช้ไม่รับรู้ถึงความซับซ้อนเบื้องหลัง

ความท้าทาย

- **Network latency** — การสื่อสารระหว่าง node ช้า
- **Partial failure** — บาง node ทำงานผิดพลาด
- **Data consistency** — การรักษาความถูกต้องของข้อมูล
- **Security** — การรักษาความปลอดภัยข้ามเครือข่าย
- **Debugging** — ยากต่อการแก้ไขปัญหา

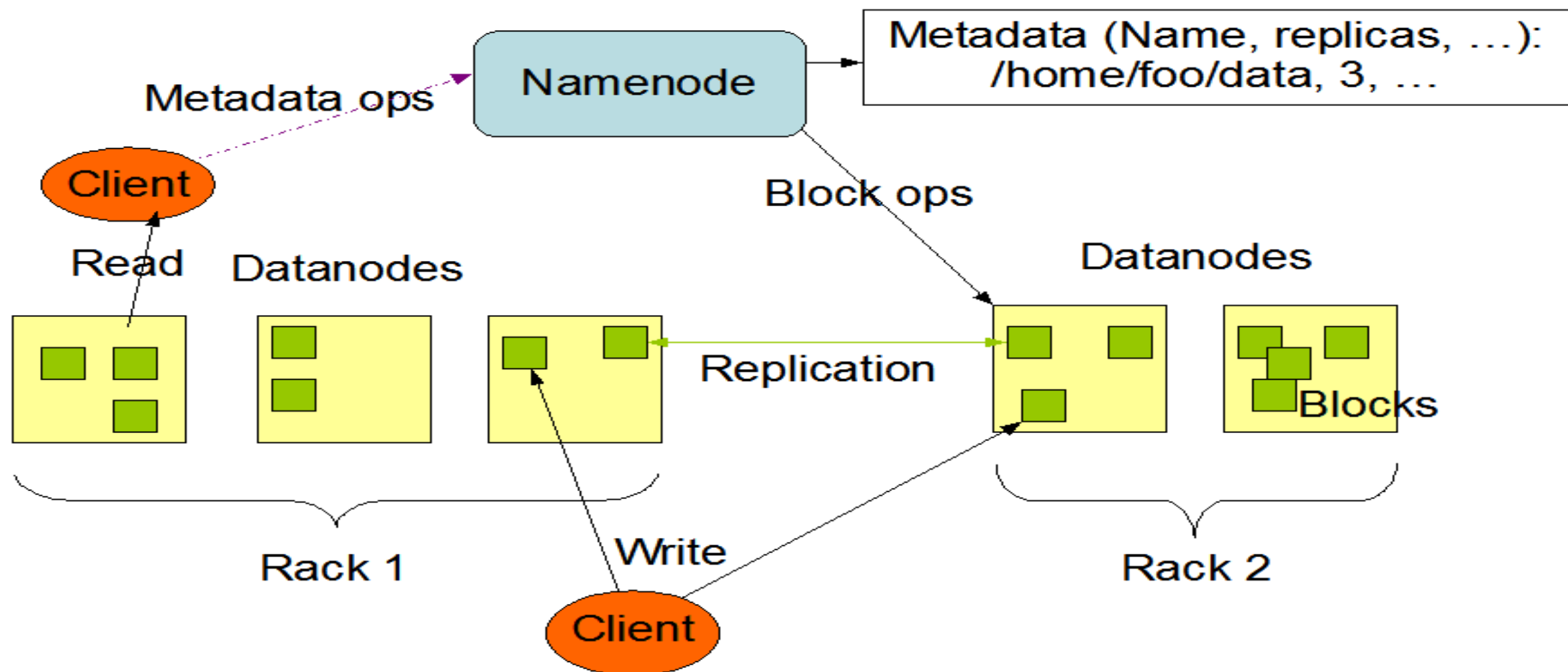
Hadoop Ecosystem

- 2006: Doug Cutting และ Mike Cafarella สร้าง Hadoop
- ได้รับแรงบันดาลใจจาก Google MapReduce และ Google File System
- กลายเป็นมาตรฐานของ Big Data ในยุคแรก

HDFS (Hadoop Distributed File System)

- **Master/Slave Architecture**
 - **NameNode** — เก็บ metadata
 - **DataNode** — เก็บข้อมูลจริง
- ไฟล์ถูกแบ่งเป็น **blocks** (128 MB default)
- **Replication** — แต่ละ block มี 3 สำเนา
- ออกแบบมาสำหรับ **write-once, read-many**

HDFS Architecture



MapReduce

- The processing engine of Hadoop ecosystem
- **Map phase** — แบ่งงาน → ประมวลผลแบบขนาน
- **Shuffle phase** — จัดกลุ่มข้อมูลตาม Key
- **Reduce phase** — รวมผลลัพธ์

Input → [Split] → Map → Shuffle → Reduce → Output

ข้อจำกัด

- เขียนงานยาก (ต้องเขียน Map/Reduce ทุกครั้ง)
- ช้าเพราะใช้ disk I/O จำนวนมาก
- ไม่เหมาะกับงาน iterative

Hadoop Ecosystem Components

Component	บทบาท
HDFS	ระบบไฟล์แบบกระจาย
MapReduce	ระบบประมวลผล batch
YARN	ทรัพยากรจัดการ cluster
Hive	Data warehouse (SQL-like)
HBase	NoSQL database
Pig	Data flow scripting
Sqoop	นำเข้าข้อมูลจาก RDBMS (Structure)
Flume	รวบรวม log (Unstructure)
Oozie	Job scheduler
ZooKeeper	Coordination service

02

การประมวลผลข้อมูลด้วย Apache Spark

Apache Spark คืออะไร?

- **Unified analytics engine** สำหรับ Big Data
- พัฒนาต่อจาก Hadoop MapReduce
- **เร็วกว่า MapReduce 10-100x** (in-memory processing)
- รองรับภาษา **Scala, Python, Java, R**

จุดเด่น

- In-memory computation
- รองรับงานหลายประเภท (batch, streaming, SQL, ML, Graph)

Spark vs MapReduce

คุณสมบัติ	MapReduce	Spark
Processing	Disk-based	In-memory
Speed	ช้า (I/O-heavy)	เร็ว (10-100x)
API	ซับซ้อน	ง่ายกว่า (DataFrame, SQL)
Real-time	ไม่รองรับ	รองรับ (Streaming)
ML/Graph	External	Built-in

Spark Stack

Spark SQL and
DataFrames

Pandas API on
Spark

Structured
Streaming

Machine
Learning
MLlib

Spark Core and RDDs

DataFrame API

- **High-level API** คล้าย pandas/tables
- มี **Schema** — type-safe
- รองรับหลาย source: CSV, JSON, Parquet, JDBC

```
# data: https://github.com/ekaratnida/Data-Engineer/blob/main/de/dataset/sample_data.csv
# setup and test pyspark on Databrick
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("DataEng Workshop") \
    .getOrCreate()
print("Spark version:", spark.version)
```



```
df = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("delimiter", ",") \
    .load("/Workspace/Users/ekarat@as.nida.ac.th/Drafts/sample_data.csv")
df.show(5)
```

```
+-----+-----+-----+-----+
|      name|amount|  category|price|qty|
+-----+-----+-----+-----+
|   Notebook|   1200|Electronics|  1200|   1|
|Wireless Mouse|   450|Electronics|   150|   3|
|   Desk Chair|  2500|  Furniture|  2500|   1|
|   Coffee Mug|   320|   Kitchen|    80|   4|
| Water Bottle|  1050|   Kitchen|   350|   3|
+-----+-----+-----+-----+
```

DataFrame — Basic Operations

```
from pyspark.sql.functions import sum, avg
df.select("name", "amount").show()
df.filter(df["amount"] > 1000).show()
df.groupBy("category").agg(
    sum("amount").alias("total"),
    avg("amount").alias("avg")
).show()
df.withColumn("total", df["price"] * df["qty"]).show()
```

Spark SQL

- เขียน SQL queries กับ DataFrame โดยตรง

```
df.createOrReplaceTempView("sales") # สร้าง temp view
result = spark.sql("""
    SELECT category, SUM(amount) as total, COUNT(*) as count
    FROM sales
    WHERE amount > 0
    GROUP BY category
    ORDER BY total DESC
""")
result.show()
```

RDD — Resilient Distributed Dataset

- **RDD** an immutable distributed collection of elements of your data, partitioned across nodes in your cluster that can be operated in parallel with a low-level API that offers transformations and actions.

การสร้าง RDD

```
# จาก collection
rdd1 = sc.parallelize([1, 2, 3, 4, 5])

# จากไฟล์
rdd2 = sc.textFile("hdfs://data/sales.csv")
```

RDD — Transformations & Actions

Transformations (Lazy)

```
rdd = sc.textFile("data.txt")  
rdd2 = rdd.map(lambda x: x.split(",")).filter(lambda x: int(x[1]) > 100)
```

Actions (Eager — trigger computation)

```
result = rdd2.collect()    # ดึงทั้งหมด  
count  = rdd2.count()      # นับจำนวน  
first  = rdd2.first()      # ตัวแรก  
top5   = rdd2.take(5)      # 5 ตัวแรก
```

```
# Convert DataFrame to RDD
df_rdd = df.rdd

print("--- Original RDD (first 5 elements) ---")
print(df_rdd.take(5))

# Example 1: Using map to extract specific columns (name, amount)
# Each row in the RDD is a Row object, you can access elements by index or name
mapped_rdd = df_rdd.map(lambda row: (row.name, row.amount))

print("\n--- Mapped RDD (name, amount - first 5 elements) ---")
for row in mapped_rdd.take(5):
    print(row)
```

```
# Example 2: Using filter to get items with amount > 1000
filtered_rdd = df_rdd.filter(lambda row: row.amount > 1000)

print("\n--- Filtered RDD (amount > 1000 - first 5 elements) ---")
for row in filtered_rdd.take(5):
    print(row)
```

```
# Example 3: Combining map and filter, then collecting results as a list
high_value_items = df_rdd \
    .filter(lambda row: row.category == 'Electronics' and row.amount > 1000) \
    .map(lambda row: f"Item: {row.name}, Amount: {row.amount}, Category: {row.category}") \
    .collect()

print("\n--- High Value Electronics Items (first 5) ---")
for item in high_value_items[:5]:
    print(item)
```


การเพิ่มประสิทธิภาพการประมวลผลเบื้องต้น (1/2)

1. Data Optimization

- Use columnar formats like Parquet or ORC.
- Data Partitioning.
- Bucketing for frequently joined tables. This can significantly reduce shuffle during joins.

2. Code & Application Optimization

- Try to reduce operations that trigger shuffles, such as `groupByKey`, `reduceByKey`, `join`, `repartition`.
- Use wider transformations (like `reduceByKey`) carefully, as they involve shuffles. Narrow transformations (like `map`, `filter`) are generally faster.
- Caching/Persisting RDDs/DataFrames: If an RDD or DataFrame is used multiple times, `cache()` or `persist()` it in memory (or disk) to avoid recomputing it.

การเพิ่มประสิทธิภาพการประมวลผลเบื้องต้น (2/2)

3. Monitoring and Profiling:

- Use the Spark Web UI to monitor your jobs, identify bottlenecks, and analyze execution plans.
- Review Spark logs for errors or warnings that might indicate performance issues.

03

แพลตฟอร์มสมัยใหม่

Databricks

- <https://www.databricks.com/>
- ก่อตั้งโดย **ผู้สร้าง Apache Spark** (UC Berkeley)
- **Managed Spark Platform** — ไม่ต้องดูแล cluster เอง
- **Delta Lake** — ACID transactions บน data lake
- **MLflow** — จัดการ Machine Learning lifecycle
- **Unity Catalog** — Data governance

จุดเด่น

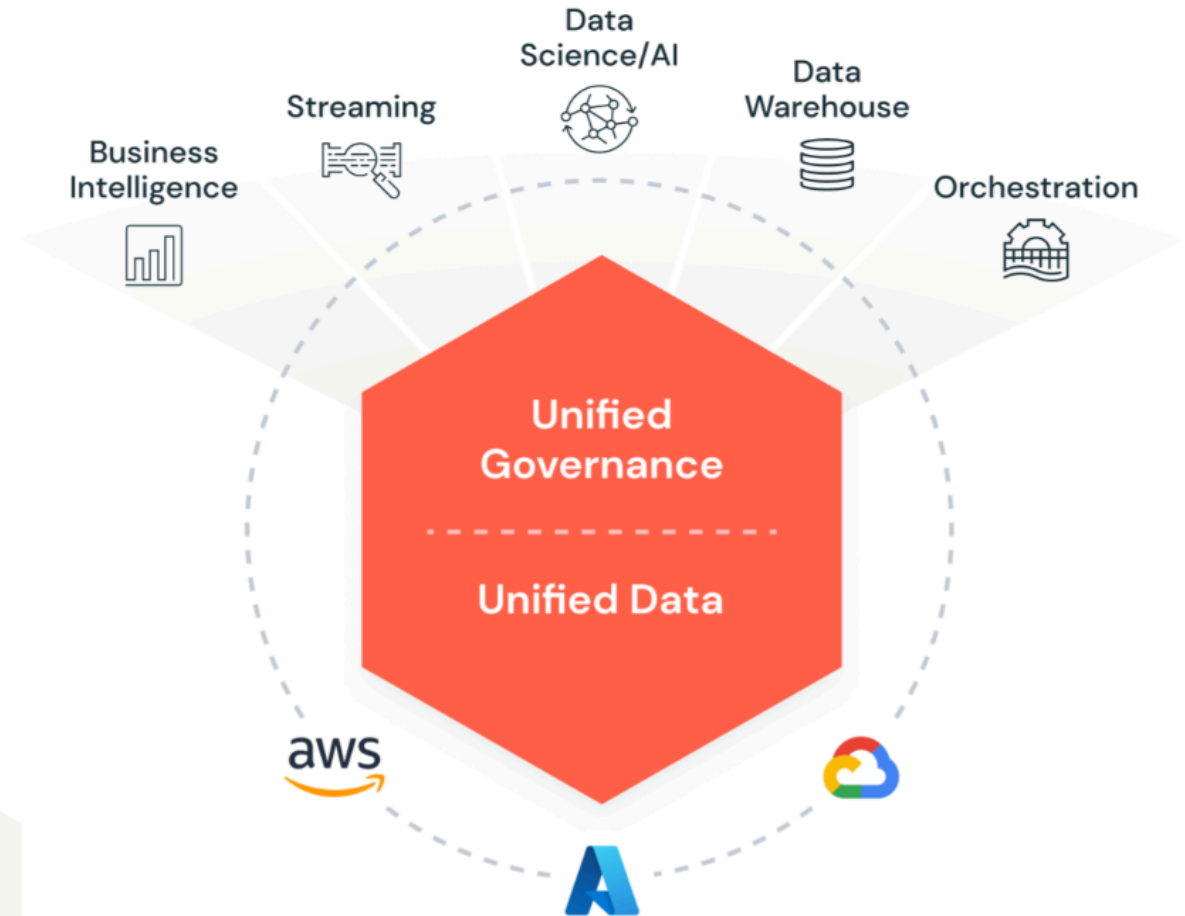
- Notebook-based (collaborative)
- Auto-scaling cluster
- Optimized Spark engine (Photon)
- SQL Analytics (SQL Warehouse)

Databricks Architecture

Unified. Open. Scalable.

The [Databricks Data Intelligence Platform](#) is built on lakehouse architecture, which combines the best elements of data lakes and data warehouses to help you reduce costs and deliver on your data and AI initiatives faster.

Built on open source and open standards, a lakehouse simplifies your data estate by eliminating the silos that historically complicate data and AI.



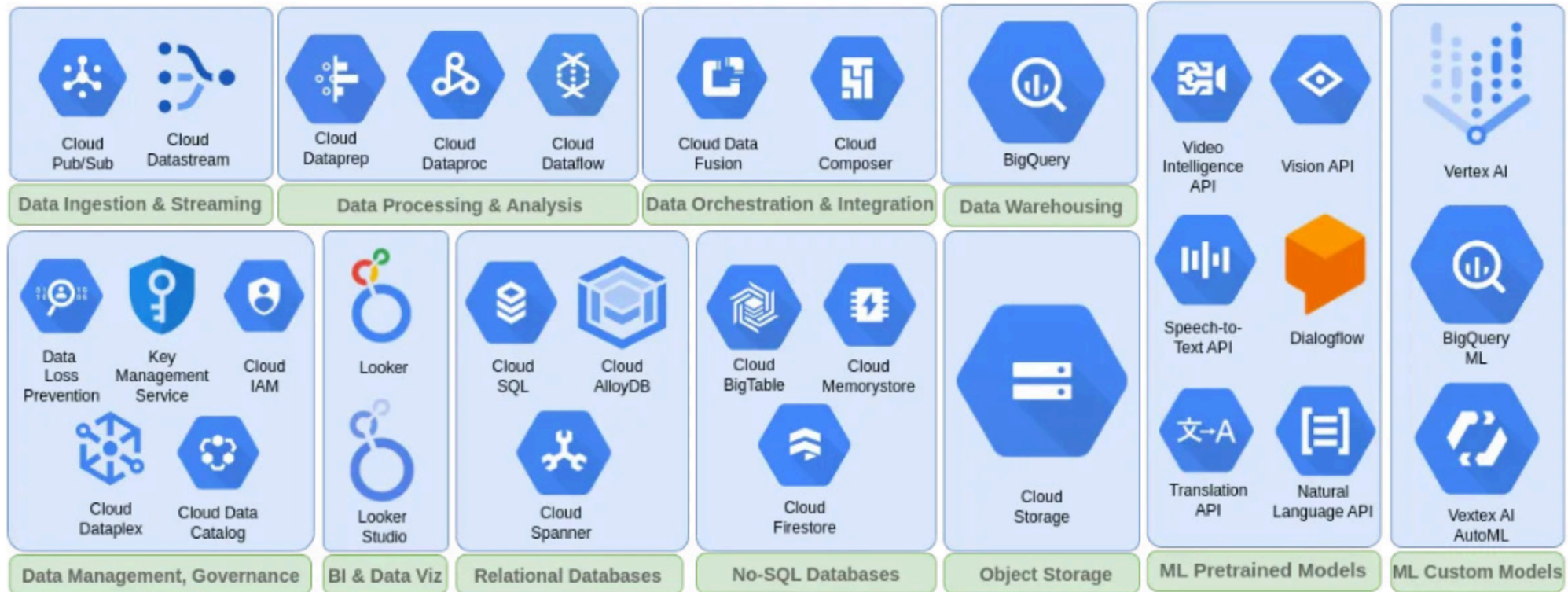
Google BigQuery

- WWW: <https://docs.cloud.google.com/bigquery/docs/sandbox>
- **Serverless Data Warehouse** ของ Google Cloud
- **No cluster management** — ง่ายตามการใช้งาน
- **Columnar storage** (Capacitor)
- **Petabyte-scale** queries ในไม่กี่วินาที

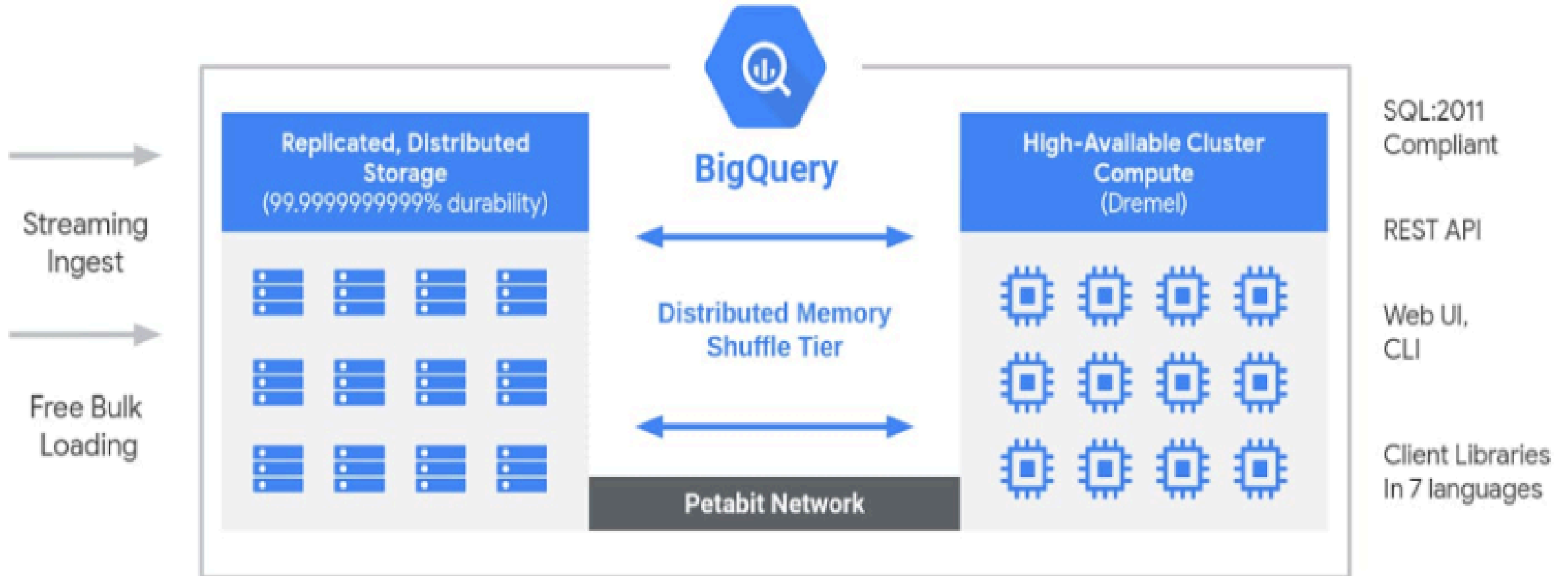
จุดเด่น

- ไม่ต้อง provision หรือ manage infrastructure
- ง่ายตาม **data scanned** (\$5/TB)
- BI integration (Looker, Tableau)
- รองรับ **Standard SQL**
- Machine learning ใน SQL (BigQuery ML)

BigQuery — Ecosystem



BigQuery - Architecture



Databricks vs BigQuery

คุณสมบัติ	Databricks	BigQuery
Architecture	Managed Spark	Serverless
Scaling	Auto-scaling cluster	Elastic (auto)
Query Engine	Spark + Photon	Dremel
Pricing	DBU-based	Data scanned
Open Source	Spark, Delta, MLflow	Proprietary
ETL	Notebook, Jobs	SQL + Dataform
ML	MLflow, AutoML	BigQuery ML
Governance	Unity Catalog	IAM + Data Catalog

เมื่อไหร่ควรใช้อะไร?

Databricks

- ต้องการควบคุม Spark configuration
- Data Engineers และ Data Scientists
- ทำงานกับ structured และ unstructured data
- ต้องการ Custom ML pipeline

BigQuery

- ต้องการ Query เร็ว ๆ ไม่ต้องจัดการ cluster
- ทึมคุ้นเคย SQL
- งาน BI / Dashboard / Reporting
- งบประมาณจำกัด (จ่ายเท่าที่ใช้)

สรุป

- **Distributed Computing** — พื้นฐานของ Big Data
- **Hadoop** — วางรากฐาน ecosystem (HDFS, MapReduce)
- **Spark** — ก้าวข้ามข้อจำกัดของ MapReduce (เร็วขึ้น 10-100x)
 - RDD → พื้นฐาน
 - DataFrame → สะดวก, เร็ว, มี optimizer
- **แพลตฟอร์มยุคใหม่** — Databricks (managed Spark) / BigQuery (serverless)

คำถาม?

Questions & Discussion

ผศ.ดร. เอกรัฐ รัชกาญจน์

ekarat@as.nida.ac.th

ekaratnida.github.io

คณะสถิติประยุกต์ · สถาบันบัณฑิตพัฒนบริหารศาสตร์ (นิด้า)